



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 7

“Contador de 00 a 99 activado por IR”

Grupo: 6CM1

Integrantes

- **García Escamilla Bryan Alexis**
- **Meléndez Macedonio Rodrigo**

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 20 de febrero de 2025

INTRODUCCIÓN

Contador 00 a 99 (Cascada de Décadas)

Un contador de 00 a 99 no es un solo componente, sino la unión de dos contadores de década (0 a 9) configurados en un arreglo llamado cascada. Su funcionamiento se basa en la transferencia de un pulso de acarreo.

El proceso de Acarreo

- **Unidades:** El primer contador recibe los pulsos externos (en este caso, del sensor infrarrojo). Cuenta de 0 a 9 de forma normal.
- **Decenas:** El segundo contador permanece en reposo hasta que el primero llega a su límite.
- **La transición:** Cuando el contador de unidades pasa de 9 a 0, genera un pulso de salida llamado Carry Out o Terminal Count. Este pulso se conecta directamente a la entrada de reloj (Clock) del contador de decenas.

Sincronización

En sistemas profesionales es preferible usar un conteo síncrono. Esto significa que ambos contadores comparten la misma señal de reloj, pero el de las decenas solo se "habilita" para aumentar cuando el de las unidades indica que ha completado su ciclo. Esto evita errores de lectura durante el cambio de 09 a 10 o de 19 a 20.

Sensores Infrarrojos

En el área de electrónica, los infrarrojos (IR) se utilizan como un interruptor sin contacto. Un sistema básico se compone de dos partes: un emisor y un receptor.

Principio de Funcionamiento

El emisor (un LED IR) lanza una luz invisible para el ojo humano. El receptor (un fototransistor o un fotodiodo) detecta esa luz y permite el paso de corriente. Para un contador, existen dos configuraciones principales:

- **Barrera (Transmisivo):** El emisor y el receptor están frente a frente. Cuando un objeto pasa entre ellos, corta el haz de luz. Ese "corte" genera el flanco (subida o bajada) que el contador interpreta como un evento.
- **Reflexivo:** El emisor y el receptor están uno al lado del otro mirando en la misma dirección. El haz de luz rebota en el objeto y regresa al receptor.

Integración

Si se unen ambos conceptos en un ATMEGA8535:

- El receptor IR se conecta a un pin de interrupción externa o al pin de entrada del Timer (T1).
- Cada vez que el haz IR se interrumpe, el software aumenta una variable.
- El microcontrolador se encarga de separar esa variable en "Decenas" y "Unidades" (usando divisiones y residuos) para enviarlas a los displays.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 7, el circuito a armar es el siguiente:

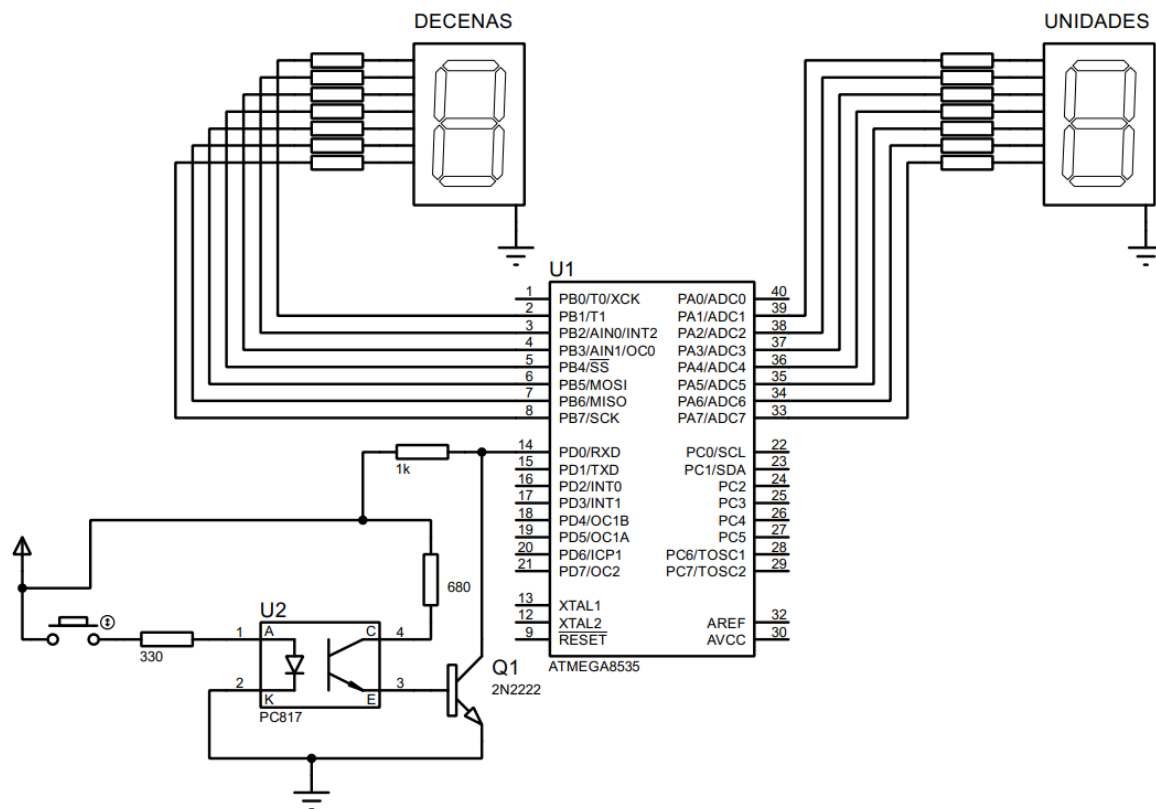


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen del circuito, el registro B se configura como de salida (PB1– PB7), el A también (PA1– PA7), mientras que el registro D como de entrada (PD0), activando la resistencia de pull-up.

Ports Settings

PORTA Port B Port C Port D

	Data Direction	Pullup/Output Value	
Bit 0	In	T	Bit 0
Bit 1	Out	0	Bit 1
Bit 2	Out	0	Bit 2
Bit 3	Out	0	Bit 3
Bit 4	Out	0	Bit 4
Bit 5	Out	0	Bit 5
Bit 6	Out	0	Bit 6
Bit 7	Out	0	Bit 7

Imagen 2. Configuraci3n del puerto A.

Ports Settings

PORTA Port B Port C Port D

	Data Direction	Pullup/Output Value	
Bit 0	In	T	Bit 0
Bit 1	Out	0	Bit 1
Bit 2	Out	0	Bit 2
Bit 3	Out	0	Bit 3
Bit 4	Out	0	Bit 4
Bit 5	Out	0	Bit 5
Bit 6	Out	0	Bit 6
Bit 7	Out	0	Bit 7

Imagen 3. Configuraci3n del puerto B.

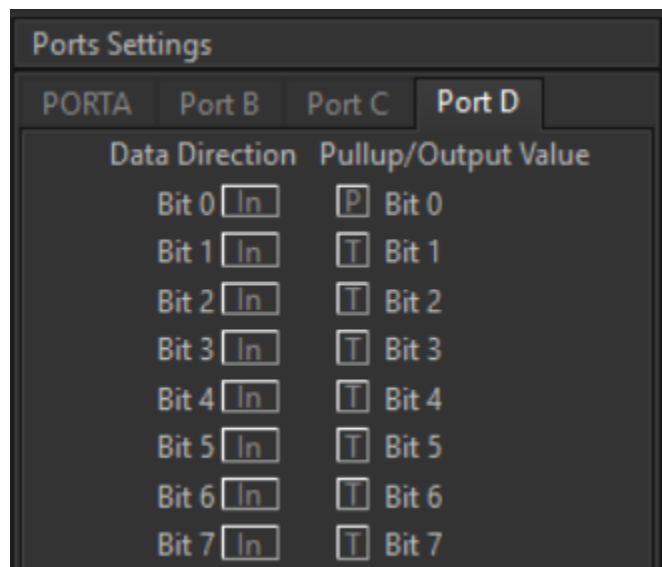


Imagen 4. Configuración del puerto D.

Para poder hacer el contador es necesario interpretar el número correspondiente del display como un número hexadecimal, para ello se realizó la siguiente tabla, tomando en cuenta lo que se indicó en la práctica 3:

Número Display	Combinaciones								Valor hexadecimal
	g	f	e	d	c	b	a		
0	0	1	1	1	1	1	1	0	0x7e
1	0	0	0	0	1	1	0	0	0x0c
2	1	0	1	1	0	1	1	0	0xb6
3	1	0	0	1	1	1	1	0	0x9e
4	1	1	0	0	1	1	0	0	0xcc
5	1	1	0	1	1	0	1	0	0xda
6	1	1	1	1	1	0	1	0	0xfa
7	0	0	0	0	1	1	1	0	0x0e
8	1	1	1	1	1	1	1	0	0xfe
9	1	1	0	1	1	1	1	0	0xde
A	1	1	1	0	1	1	1	0	0xee
B	1	1	1	1	1	0	0	0	0xf8
C	0	1	1	1	0	0	1	0	0x72
D	1	0	1	1	1	1	0	0	0xbc
E	1	1	1	1	0	0	1	0	0xf2
F	1	1	1	0	0	0	1	0	0xe2

Tabla 1. Representación hexadecimal para los números del 0 al 9 y del A a la F en hexadecimal.

Código del circuito de CodeVision:

```

1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4       Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr  ctica 7 'Contador de 00 a 99'
8  Version : 1.0
9  Date    : 16/02/2026
10 Author  : Garc  a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type      : ATmega8535
16 Program type   : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model   : Small
19 External RAM size : 0
20 Data Stack size : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #include <delay.h>
26 #define BOTON PIND.0

```

```

28 // Declare your global variables here
29
30 void main(void) {
31     // Declare your Local variables here
32     bit estado_boton_anterior, estado_boton_actual;
33     unsigned char unidad, decena;
34     const char bcd[10] = {0x7e, 0x0c, 0xb6, 0x9e, 0xcc, 0xda, 0xfa, 0x0e, 0xfe, 0xde};
35
36     // Input/Output Ports initialization
37     // Port A initialization
38     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=In
39     DDRA=(1<<DDA7) | (1<<DDA6) | (1<<DDA5) | (1<<DDA4) | (1<<DDA3) | (1<<DDA2) | (1<<DDA1) | (0<<DDA0);
40     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=1
41     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
42
43     // Port B initialization
44     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=In
45     DDRB=(1<<ddb7) | (1<<ddb6) | (1<<ddb5) | (1<<ddb4) | (1<<ddb3) | (1<<ddb2) | (1<<ddb1) | (0<<ddb0);
46     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=1
47     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
48
49     // Port C initialization
50     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
51     DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
52     // State: Bit7=1 Bit6=1 Bit5=1 Bit4=1 Bit3=1 Bit2=1 Bit1=1 Bit0=1
53     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

```

```

55 // Port D initialization
56 // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
57 DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
58 // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=P
59 PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (1<<PORTD0);
60
61 // Timer/Counter 0 initialization
62 // Clock source: System Clock
63 // Clock value: Timer 0 Stopped
64 // Mode: Normal top=0xFF
65 // OC0 output: Disconnected
66 TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
67 TCNT0=0x00;
68 OCR0=0x00;

```

```

70 // Timer/Counter 1 initialization
71 // Clock source: System Clock
72 // Clock value: Timer1 Stopped
73 // Mode: Normal top=0xFFFF
74 // OC1A output: Disconnected
75 // OC1B output: Disconnected
76 // Noise Canceler: Off
77 // Input Capture on Falling Edge
78 // Timer1 Overflow Interrupt: Off
79 // Input Capture Interrupt: Off
80 // Compare A Match Interrupt: Off
81 // Compare B Match Interrupt: Off
82 TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
83 TCCR1B=(0<<TCNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
84 TCNT1H=0x00;
85 TCNT1L=0x00;
86 ICR1H=0x00;
87 ICR1L=0x00;
88 OCR1AH=0x00;
89 OCR1AL=0x00;
90 OCR1BH=0x00;
91 OCR1BL=0x00;

```

```

93 // Timer/Counter 2 initialization
94 // Clock source: System Clock
95 // Clock value: Timer2 Stopped
96 // Mode: Normal top=0xFF
97 // OC2 output: Disconnected
98 ASSR=0<<AS2;
99 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
100 TCNT2=0x00;
101 OCR2=0x00;
102
103 // Timer(s)/Counter(s) Interrupt(s) initialization
104 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
105
106 // External Interrupt(s) initialization
107 // INT0: Off
108 // INT1: Off
109 // INT2: Off
110 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
111 MCUCSR=(0<<ISC2);
112
113 // USART initialization
114 // USART disabled
115 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);

```

```

117 // Analog Comparator initialization
118 // Analog Comparator: Off
119 // The Analog Comparator's positive input is
120 // connected to the AIN0 pin
121 // The Analog Comparator's negative input is
122 // connected to the AIN1 pin
123 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
124 SFIOR=(0<<ACME);
125
126 // ADC initialization
127 // ADC disabled
128 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
129
130 // SPI initialization
131 // SPI disabled
132 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
133
134 // TWI initialization
135 // TWI disabled
136 TWCR=(0<<TWIE) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

```

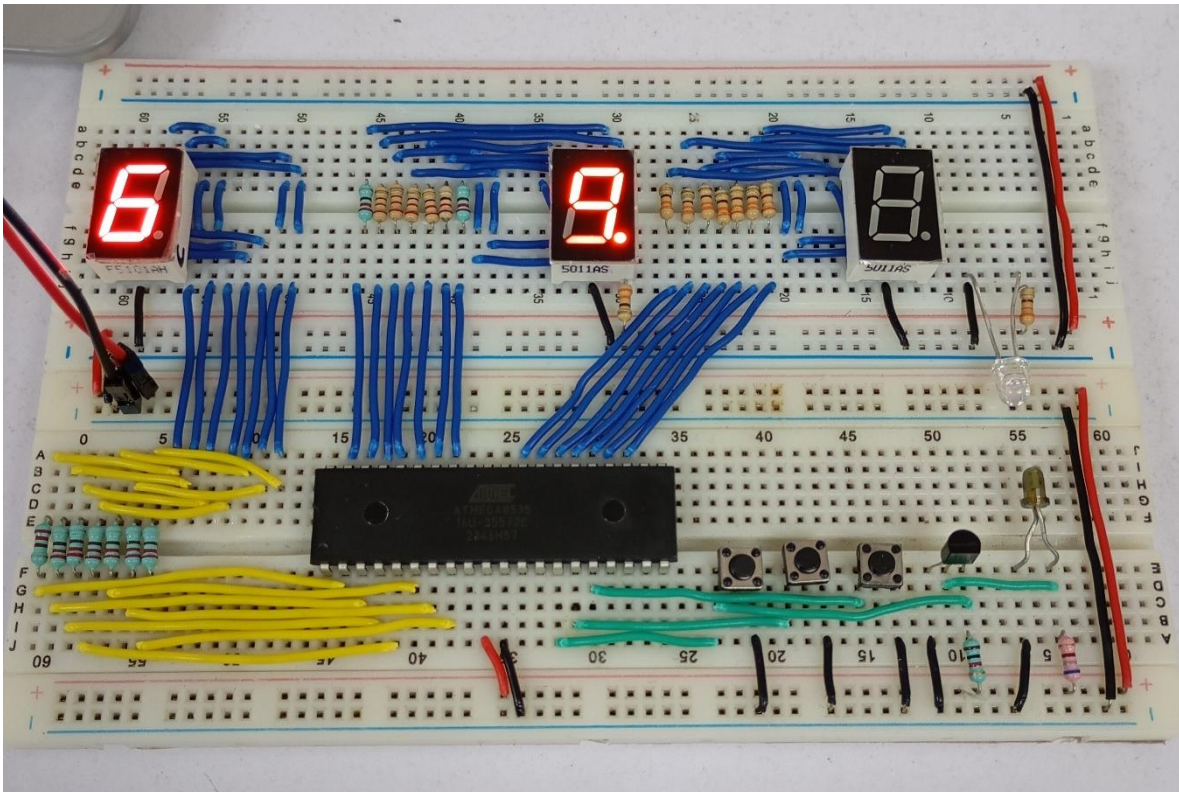
```

138 while (1) {
139     // Place your code here
140     if (!BOTON) {
141         estado_boton_actual = 0;
142     } else {
143         estado_boton_actual = 1;
144     }
145
146     // Si se detecta un flanco de subida de 1 a 0
147     if (estado_boton_actual && !estado_boton_anterior) {
148         unidad++;
149         delay_ms(40);
150     }
151
152     // Si se detecta un flanco de bajada de 0 a 1
153     if (!estado_boton_actual && estado_boton_anterior) {
154         delay_ms(40);
155     }
156
157     // Si la unidad llega a 10 se reinicia y se incrementa la decena
158     if (unidad == 10) {
159         unidad = 0;
160         decena++;
161     }
162
163     // Si la decena llega a 10, se llega a 99 + 1, este se reinicia
164     if (decena == 10) {
165         decena = 0;
166     }

```

```
168 // Obtener los números hexadecimales
169 PORTA = bcd[unidad];
170 PORTB = bcd[decena];
171
172 estado_boton_anterior = estado_boton_actual;
173 }
174 }
```

Circuito físico



CONCLUSIÓN

- **García Escamilla Bryan Alexis**

En esta práctica se pudo observar la aplicación de la estrategia que se usó para resolver el problema de los botones de las prácticas anteriores, solo que, a diferencia de estas, aquí se aplicó a un par infrarrojo, esto hizo que el contador funcionara correctamente incluso cuando se usó un fidget spinner para incrementar el contador rápidamente.

Por otro lado, en esta práctica se aprendió a programar un contador de 00 a 99 usando dos displays de forma independiente, sin embargo, el problema con esto es que se ocupan dos registros completos del microcontrolador lo que nos deja con poco espacio para demás dispositivos, por lo que más adelante se verá como se pueden multiplexar hasta 6 displays y controlarlos usando solo dos registros.

Aquí se presentó un problema común al usar el par infrarrojo, pues la posición del emisor (led infrarrojo) y del receptor (fotodiodo) deben de coincidir en altura de lo contrario la luz infrarroja no llega correctamente al receptor, por tanto, no se produce una salida en este y eventualmente el conteo no se realiza.

- **Meléndez Macedonio Rodrigo**

Para esta práctica se desarrolló un contador de 00 a 99 usando dos displays de manera independiente bajo la misma lógica de programación que las prácticas de contadores anteriores usando el botón, solo que en este caso en lugar de usar un botón usamos un par infrarrojo. Este funciona conectando en el circuito un emisor y un receptor que deben estar posicionados uno frente al otro si o si para que el receptor pueda recibir correctamente la luz infrarroja y emitir la señal.

Sin embargo, durante el desarrollo de esta práctica se presentaron un par de problemas en esta parte, ya que había veces en las que los pares infrarrojos no estaban bien posicionados y esto impedía que el contador aumentará al momento de pasar un objeto entre estos. Más allá de ese detalle, la práctica se completó exitosamente e incluso pudimos probar el circuito con un fidget spinner e incluso a esa velocidad el contador aumentaba cada vez que este giraba a gran velocidad.

BIBLIOGRAFÍA

- (s.f.). *Optical Sensors*. All About Circuits. Recuperado el 19 de febrero de 2026. <https://www.allaboutcircuits.com/technical-articles/introduction-to-optical-sensors/>